

ZetCode

[All](#) [Golang](#) [Python](#) [C#](#) [Java](#) [JavaScript](#) [Subscribe](#)



Learn More



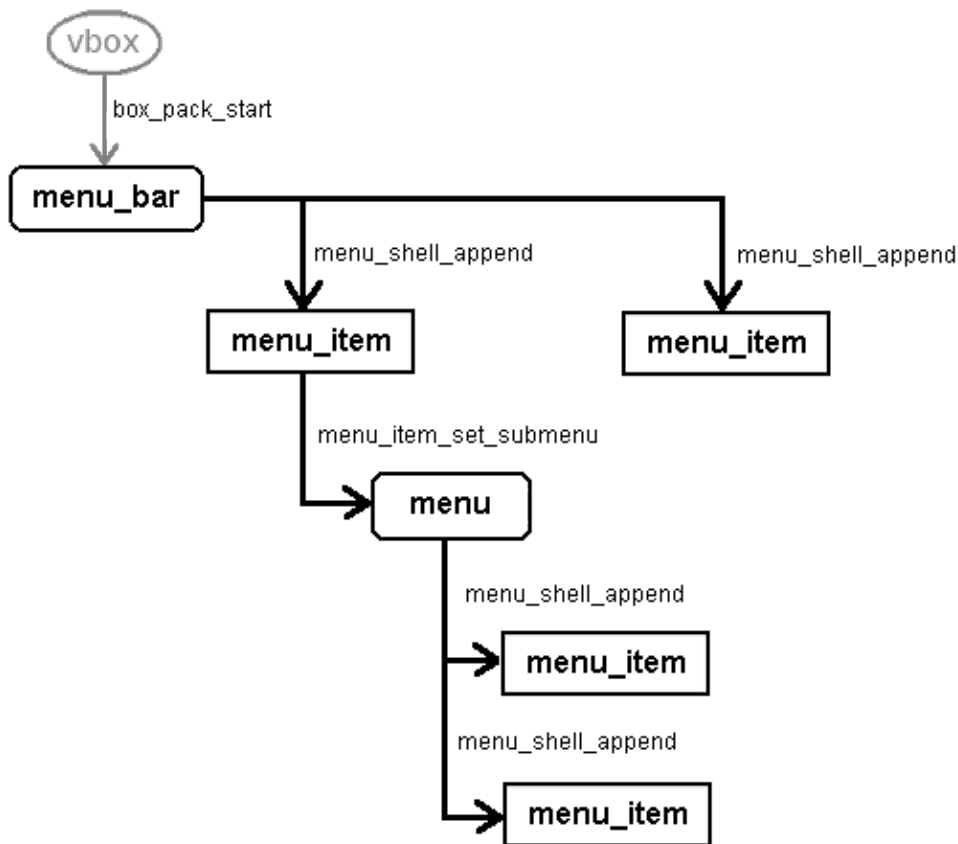
[Contents](#) [Previous](#) [Next](#)

Menus and toolbars in GTK+

last modified January 6, 2022

In this part of the GTK+ programming tutorial, we work with menus and toolbars. A *menubar* is a common part of a GUI application. It is a group of commands located in various menus.

GtkMenuBar is a widget that creates a menubar. It contains one to many GtkMenuItems. A menu item is an object which a user can select. GtkMenu implements a drop down menu consisting of a list of GtkMenuItem objects, which can be navigated and activated by the user to perform application functions. A GtkMenu is attached to the menu items of the menubar or menu items of another menu.



Advertisements

Simple menu example

In our first example, we create a menubar with one File menu.

`simplemenu.c`

```
#include <gtk/gtk.h>

int main(int argc, char *argv[]) {

    GtkWidget *window;
    GtkWidget *vbox;

    GtkWidget *menubar;
    GtkWidget *fileMenu;
    GtkWidget *fileMi;
    GtkWidget *quitMi;

    gtk_init(&argc, &argv);

    window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
    gtk_window_set_position(GTK_WINDOW(window), GTK_WIN_POS_CENTER);
    gtk_window_set_default_size(GTK_WINDOW(window), 300, 200);
    gtk_window_set_title(GTK_WINDOW(window), "Simple menu");

    vbox = gtk_vbox_new(FALSE, 0);
    gtk_container_add(GTK_CONTAINER(window), vbox);
```

```
gtk_menu_item_set_submenu(GTK_MENU_ITEM(fileMi), fileMenu);
gtk_menu_shell_append(GTK_MENU_SHELL(fileMenu), quitMi);
gtk_menu_shell_append(GTK_MENU_SHELL(menuBar), fileMi);
gtk_box_pack_start(GTK_BOX(vbox), menuBar, FALSE, FALSE, 0);

g_signal_connect(G_OBJECT(window), "destroy",
                 G_CALLBACK(gtk_main_quit), NULL);

g_signal_connect(G_OBJECT(quitMi), "activate",
                 G_CALLBACK(gtk_main_quit), NULL);

gtk_widget_show_all(window);

gtk_main();

return 0;
}
```

The menu in the example has one menu item. By selecting the item, the application quits.

Advertisements

```
menuBar = gtk_menu_bar_new();
```

The `gtk_menu_bar_new` creates a new `GtkMenuBar`.

```
fileMenu = gtk_menu_new();
```

The `gtk_menu_new` function creates a new `GtkMenu`.

```
gtk_menu_item_set_submenu(GTK_MENU_ITEM(fileMi), fileMenu);
```

The `fileMenu` is set to the File menu item with the `gtk_menu_item_set_submenu` function. Menus are containers which hold menu items. They are themselves plugged to a particular menu item.

```
gtk_menu_shell_append(GTK_MENU_SHELL(fileMenu), quitMi);
```

```
gtk_menu_shell_append(GTK_MENU_SHELL(menubar), fileMi);
```

The File menu item is added to the menubar with the `gtk_menu_shell_append` function. Both `GtkMenu` and `GtkMenuBar` are derived from the `GtkMenuShell`.

```
g_signal_connect(G_OBJECT(quitMi), "activate",  
                G_CALLBACK(gtk_main_quit), NULL);
```

By selecting the quit menu item, we terminate the application.

Advertisements

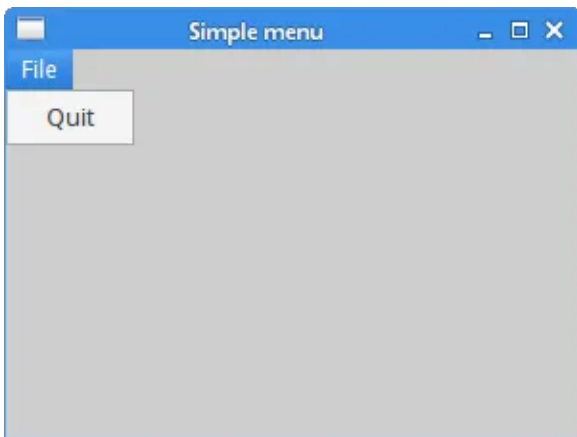


Figure: Simple menu

Submenu

The next example demonstrates how to create a submenu. A submenu is a menu inside another menu.

submenu.c

```
#include <gtk/gtk.h>  
  
int main(int argc, char *argv[]) {  
  
    GtkWidget *window;
```

```
GtkWidget *sep;
GtkWidget *fileMi;
GtkWidget *imprMi;
GtkWidget *feedMi;
GtkWidget *bookMi;
GtkWidget *mailMi;
GtkWidget *quitMi;

gtk_init(&argc, &argv);

window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
gtk_window_set_position(GTK_WINDOW(window), GTK_WIN_POS_CENTER);
gtk_window_set_default_size(GTK_WINDOW(window), 300, 200);
gtk_window_set_title(GTK_WINDOW(window), "Submenu");

vbox = gtk_vbox_new(FALSE, 0);
gtk_container_add(GTK_CONTAINER(window), vbox);

menubar = gtk_menu_bar_new();

fileMenu = gtk_menu_new();
fileMi = gtk_menu_item_new_with_label("File");

imprMenu = gtk_menu_new();
imprMi = gtk_menu_item_new_with_label("Import");
feedMi = gtk_menu_item_new_with_label("Import news feed...");
bookMi = gtk_menu_item_new_with_label("Import bookmarks...");
mailMi = gtk_menu_item_new_with_label("Import mail...");

gtk_menu_item_set_submenu(GTK_MENU_ITEM(imprMi), imprMenu);
gtk_menu_shell_append(GTK_MENU_SHELL(imprMenu), feedMi);
gtk_menu_shell_append(GTK_MENU_SHELL(imprMenu), bookMi);
gtk_menu_shell_append(GTK_MENU_SHELL(imprMenu), mailMi);
sep = gtk_separator_menu_item_new();
quitMi = gtk_menu_item_new_with_label("Quit");

gtk_menu_item_set_submenu(GTK_MENU_ITEM(fileMi), fileMenu);
gtk_menu_shell_append(GTK_MENU_SHELL(fileMenu), imprMi);
gtk_menu_shell_append(GTK_MENU_SHELL(fileMenu), sep);
gtk_menu_shell_append(GTK_MENU_SHELL(fileMenu), quitMi);
gtk_menu_shell_append(GTK_MENU_SHELL(menubar), fileMi);
gtk_box_pack_start(GTK_BOX(vbox), menubar, FALSE, FALSE, 0);

g_signal_connect(G_OBJECT(window), "destroy",
                 G_CALLBACK(gtk_main_quit), NULL);

g_signal_connect(G_OBJECT(quitMi), "activate",
                 G_CALLBACK(gtk_main_quit), NULL);
```

```
}

```

The example creates a menu inside another menu. The submenu has three menu items. We also add a horizontal separator.

Advertisements

```
imprMenu = gtk_menu_new();
imprMi = gtk_menu_item_new_with_label("Import");
feedMi = gtk_menu_item_new_with_label("Import news feed...");
bookMi = gtk_menu_item_new_with_label("Import bookmarks...");
mailMi = gtk_menu_item_new_with_label("Import mail...");

```

This is a submenu with its menu items.

```
gtk_menu_item_set_submenu(GTK_MENU_ITEM(imprMi), imprMenu);

```

The imprMenu submenu is added to its own menu item.

```
gtk_menu_shell_append(GTK_MENU_SHELL(imprMenu), feedMi);
gtk_menu_shell_append(GTK_MENU_SHELL(imprMenu), bookMi);
gtk_menu_shell_append(GTK_MENU_SHELL(imprMenu), mailMi);

```

The three menu items are added to the submenu with the `gtk_menu_shell_append` function.

```
sep = gtk_separator_menu_item_new();

```

A horizontal menu separator is created with the `gtk_separator_menu_item_new` function.

```
gtk_menu_shell_append(GTK_MENU_SHELL(fileMenu), imprMi);
gtk_menu_shell_append(GTK_MENU_SHELL(fileMenu), sep);

```

The imprMi and the separator are added to the File menu with the `gtk_menu_shell_append` function

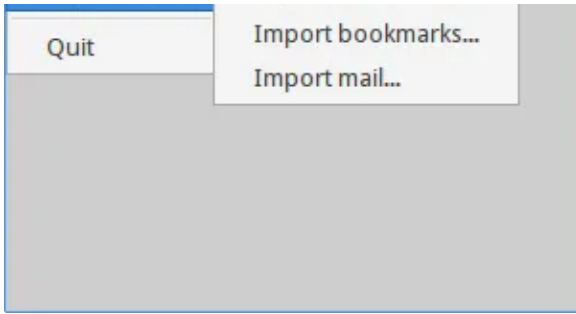


Figure: Submenu

Image menus, mnemonics & accelerators

`GtkImageMenuItem` is a menu item which has an icon next to the text label. Since the user can disable displaying of menu icons, we still need to fill in the text label. *Accelerators* are keyboard shortcuts for activating a menu item. *Mnemonics* are keyboard shortcuts for GUI elements. They are represented as underlined characters. Note that in some environments, we first need to press the mouseless modifier (usually `Alt`) to show the underlined characters.

We might also have our environment configured not to show menu images. To turn the menu images on, we launch the `gconf-editor` and go to `/desktop/gnome/interface/menus_have_icons` and check the option.

imagemenu.c

```
#include <gtk/gtk.h>
#include <gdk/gdkkeysyms.h>

int main(int argc, char *argv[]) {

    GtkWidget *window;
    GtkWidget *vbox;

    GtkWidget *menubar;
    GtkWidget *fileMenu;
    GtkWidget *fileMi;
    GtkWidget *newMi;
    GtkWidget *openMi;
    GtkWidget *quitMi;

    GtkWidget *sep;
```



```
window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
gtk_window_set_position(GTK_WINDOW(window), GTK_WIN_POS_CENTER);
gtk_window_set_default_size(GTK_WINDOW(window), 300, 200);
gtk_window_set_title(GTK_WINDOW(window), "Images");

vbox = gtk_vbox_new(FALSE, 0);
gtk_container_add(GTK_CONTAINER(window), vbox);

menubar = gtk_menu_bar_new();
fileMenu = gtk_menu_new();

accel_group = gtk_accel_group_new();
gtk_window_add_accel_group(GTK_WINDOW(window), accel_group);

fileMi = gtk_menu_item_new_with_mnemonic("_File");
newMi = gtk_image_menu_item_new_from_stock(GTK_STOCK_NEW, NULL);
openMi = gtk_image_menu_item_new_from_stock(GTK_STOCK_OPEN, NULL);
sep = gtk_separator_menu_item_new();
quitMi = gtk_image_menu_item_new_from_stock(GTK_STOCK_QUIT, accel_group);

gtk_widget_add_accelerator(quitMi, "activate", accel_group,
    GDK_q, GDK_CONTROL_MASK, GTK_ACCEL_VISIBLE);

gtk_menu_item_set_submenu(GTK_MENU_ITEM(fileMi), fileMenu);
gtk_menu_shell_append(GTK_MENU_SHELL(fileMenu), newMi);
gtk_menu_shell_append(GTK_MENU_SHELL(fileMenu), openMi);
gtk_menu_shell_append(GTK_MENU_SHELL(fileMenu), sep);
gtk_menu_shell_append(GTK_MENU_SHELL(fileMenu), quitMi);
gtk_menu_shell_append(GTK_MENU_SHELL(menubar), fileMi);
gtk_box_pack_start(GTK_BOX(vbox), menubar, FALSE, FALSE, 0);

g_signal_connect(G_OBJECT(window), "destroy",
    G_CALLBACK(gtk_main_quit), NULL);

g_signal_connect(G_OBJECT(quitMi), "activate",
    G_CALLBACK(gtk_main_quit), NULL);

gtk_widget_show_all(window);

gtk_main();

return 0;
}
```

The example has three menu items with icons. The menu items can be selected with mnemonics. The Quit menu item has a keyboard accelerator.

Advertisements

```
accel_group = gtk_accel_group_new();
gtk_window_add_accel_group(GTK_WINDOW(window), accel_group);
...
quitMi = gtk_image_menu_item_new_from_stock(GTK_STOCK_QUIT, accel_group);

gtk_widget_add_accelerator(quitMi, "activate", accel_group,
    GDK_q, GDK_CONTROL_MASK, GTK_ACCEL_VISIBLE);
```

An accelerator group is a group of keyboard accelerators, typically attached to a toplevel window. Here we create the Ctrl+Q keyboard accelerator.

```
fileMi = gtk_menu_item_new_with_mnemonic("_File");
```

The `gtk_menu_item_new_with_mnemonic` creates a menu item which can have a mnemonic. Underscores in label indicate the mnemonic for the menu item. The character is combined with the mouseless modifier, usually Alt. In our case, we have created the Alt+F mnemonic.

```
newMi = gtk_image_menu_item_new_from_stock(GTK_STOCK_NEW, NULL);
openMi = gtk_image_menu_item_new_from_stock(GTK_STOCK_OPEN, NULL);
```

The `gtk_image_menu_item_new_from_stock` creates a `GtkImageMenuItem` containing the image and text from a stock item.

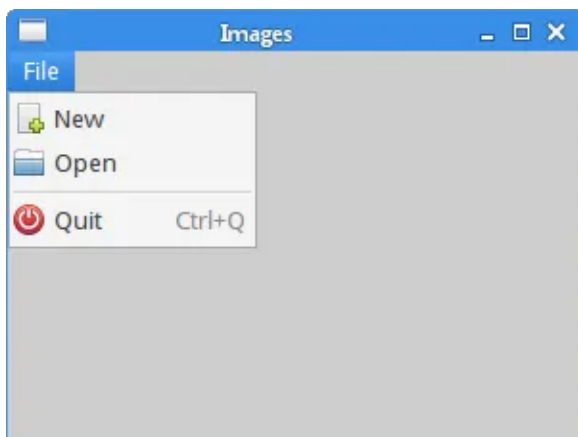


Figure: Menu items with icons

Advertisements

A `GtkCheckMenuItem` is a menu item with a check box.

`checkmenuitem.c`

```
#include <gtk/gtk.h>

void toggle_statusbar(GtkWidget *widget, gpointer statusbar) {

    if (gtk_check_menu_item_get_active(GTK_CHECK_MENU_ITEM(widget))) {

        gtk_widget_show(statusbar);
    } else {

        gtk_widget_hide(statusbar);
    }
}

int main(int argc, char *argv[]) {

    GtkWidget *window;
    GtkWidget *vbox;

    GtkWidget *menubar;
    GtkWidget *viewmenu;
    GtkWidget *view;
    GtkWidget *tog_stat;
    GtkWidget *statusbar;

    gtk_init(&argc, &argv);

    window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
    gtk_window_set_position(GTK_WINDOW(window), GTK_WIN_POS_CENTER);
    gtk_window_set_default_size(GTK_WINDOW(window), 300, 200);
    gtk_window_set_title(GTK_WINDOW(window), "GtkCheckMenuItem");

    vbox = gtk_vbox_new(FALSE, 0);
    gtk_container_add(GTK_CONTAINER(window), vbox);

    menubar = gtk_menu_bar_new();
    viewmenu = gtk_menu_new();

    view = gtk_menu_item_new_with_label("View");
    tog_stat = gtk_check_menu_item_new_with_label("View statusbar");
    gtk_check_menu_item_set_active(GTK_CHECK_MENU_ITEM(tog_stat), TRUE);

    gtk_menu_item_set_submenu(GTK_MENU_ITEM(view), viewmenu);
```

```
gtk_box_pack_end(GTK_BOX(vbox), statusbar, FALSE, TRUE, 0);

g_signal_connect(G_OBJECT(window), "destroy",
                 G_CALLBACK(gtk_main_quit), NULL);

g_signal_connect(G_OBJECT(tog_stat), "activate",
                 G_CALLBACK(toggle_statusbar), statusbar);

gtk_widget_show_all(window);

gtk_main();

return 0;
}
```

The example contains a `GtkCheckMenuItem` in a View menu. If the check box is activated, the statusbar widget is shown.

Advertisements

```
tog_stat = gtk_check_menu_item_new_with_label("View statusbar");
```

The `gtk_check_menu_item_new_with_label` function creates a new `CheckMenuItem`.

```
statusbar = gtk_statusbar_new();
```

The `gtk_statusbar_new` function creates a new `GtkStatusbar` widget. It is used to report messages of minor importance to the user.

```
if (gtk_check_menu_item_get_active(GTK_CHECK_MENU_ITEM(widget))) {

    gtk_widget_show(statusbar);
} else {

    gtk_widget_hide(statusbar);
}
```

If the check box in the menu item is activated we show the statusbar widget.

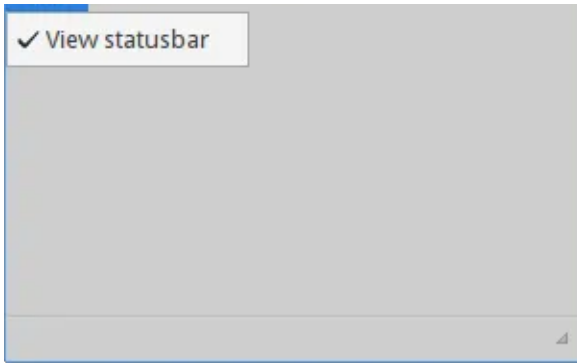


Figure: GtkCheckMenuItem

Popup menu

In the next example, we create a popup menu. A popup menu is also called a context menu. This type of menu is usually shown when we right click on a GUI object.

popupmenu.c

```
#include <gtk/gtk.h>

int show_popup(GtkWidget *widget, GdkEvent *event) {

    const gint RIGHT_CLICK = 3;

    if (event->type == GDK_BUTTON_PRESS) {

        GdkEventButton *bevent = (GdkEventButton *) event;

        if (bevent->button == RIGHT_CLICK) {

            gtk_menu_popup(GTK_MENU(widget), NULL, NULL, NULL, NULL,
                           bevent->button, bevent->time);
        }

        return TRUE;
    }

    return FALSE;
}

int main(int argc, char *argv[]) {

    GtkWidget *window;
```

```
gtk_init(&argc, &argv);

window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
gtk_window_set_position(GTK_WINDOW(window), GTK_WIN_POS_CENTER);
gtk_window_set_default_size(GTK_WINDOW(window), 300, 200);
gtk_window_set_title(GTK_WINDOW(window), "Popup menu");

ebox = gtk_event_box_new();
gtk_container_add(GTK_CONTAINER(window), ebox);

pmenu = gtk_menu_new();

hideMi = gtk_menu_item_new_with_label("Minimize");
gtk_widget_show(hideMi);
gtk_menu_shell_append(GTK_MENU_SHELL(pmenu), hideMi);

quitMi = gtk_menu_item_new_with_label("Quit");
gtk_widget_show(quitMi);
gtk_menu_shell_append(GTK_MENU_SHELL(pmenu), quitMi);

g_signal_connect_swapped(G_OBJECT(hideMi), "activate",
    G_CALLBACK(gtk_window_iconify), GTK_WINDOW(window));

g_signal_connect(G_OBJECT(quitMi), "activate",
    G_CALLBACK(gtk_main_quit), NULL);

g_signal_connect(G_OBJECT(window), "destroy",
    G_CALLBACK(gtk_main_quit), NULL);

g_signal_connect_swapped(G_OBJECT(ebox), "button-press-event",
    G_CALLBACK(show_popup), pmenu);

gtk_widget_show_all(window);

gtk_main();

return 0;
}
```

In the code example, we create a popup menu with two menu items. The first minimizes the window, and the second terminates the application.

Advertisements

In order to process button press events, we create a `GtkEventBox`.

```
pmenu = gtk_menu_new();
```

A popup menu is a `GtkMenu`.

```
hideMi = gtk_menu_item_new_with_label("Minimize");  
gtk_widget_show(hideMi);  
gtk_menu_shell_append(GTK_MENU_SHELL(pmenu), hideMi);
```

The first menu item is added to the popup menu.

```
g_signal_connect_swapped(G_OBJECT(hideMi), "activate",  
    G_CALLBACK(gtk_window_iconify), GTK_WINDOW(window));
```

Selecting the first menu item minimizes the window. We connect the activate signal of the Hide menu item to the `gtk_window_iconify` function. The term iconify is a synonym for minimize.

```
g_signal_connect_swapped(G_OBJECT(ebox), "button_press_event",  
    G_CALLBACK(show_popup), pmenu);
```

When a mouse button is pressed, a button-press-event signal is emitted. We connect the signal to the `show_popup` function and pass it the popup menu.

Advertisements

```
if (event->type == GDK_BUTTON_PRESS) {
```

Inside the event handler, we check for the button press event type.

```
if (bevent->button == RIGHT_CLICK) {  
  
    gtk_menu_popup(GTK_MENU(widget), NULL, NULL, NULL, NULL,  
        bevent->button, bevent->time);  
}
```

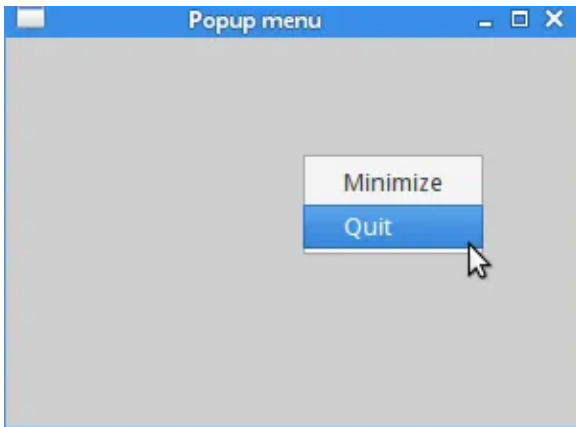


Figure: Popup menu

Advertisements

A toolbar

Menus group commands that we can use in application. Toolbars provide a quick access to the most frequently used commands. `GtkToolbar` is a toolbar widget in GTK+. A toolbar can contain instances of a subclass of a `GtkToolItem`, e.g. a `GtkToolButton` or a `GtkSeparatorToolItem`.

toolbar.c

```
#include <gtk/gtk.h>

int main(int argc, char *argv[]) {

    GtkWidget *window;
    GtkWidget *vbox;

    GtkWidget *toolbar;
    GtkToolItem *newTb;
    GtkToolItem *openTb;
    GtkToolItem *saveTb;
    GtkToolItem *sep;
    GtkToolItem *exitTb;

    gtk_init(&argc, &argv);
```



```
vbox = gtk_vbox_new(FALSE, 0);
gtk_container_add(GTK_CONTAINER(window), vbox);

toolbar = gtk_toolbar_new();
gtk_toolbar_set_style(GTK_TOOLBAR(toolbar), GTK_TOOLBAR_ICONS);

newTb = gtk_tool_button_new_from_stock(GTK_STOCK_NEW);
gtk_toolbar_insert(GTK_TOOLBAR(toolbar), newTb, -1);

openTb = gtk_tool_button_new_from_stock(GTK_STOCK_OPEN);
gtk_toolbar_insert(GTK_TOOLBAR(toolbar), openTb, -1);

saveTb = gtk_tool_button_new_from_stock(GTK_STOCK_SAVE);
gtk_toolbar_insert(GTK_TOOLBAR(toolbar), saveTb, -1);

sep = gtk_separator_tool_item_new();
gtk_toolbar_insert(GTK_TOOLBAR(toolbar), sep, -1);

exitTb = gtk_tool_button_new_from_stock(GTK_STOCK_QUIT);
gtk_toolbar_insert(GTK_TOOLBAR(toolbar), exitTb, -1);

gtk_box_pack_start(GTK_BOX(vbox), toolbar, FALSE, FALSE, 5);

g_signal_connect(G_OBJECT(exitTb), "clicked",
                 G_CALLBACK(gtk_main_quit), NULL);

g_signal_connect(G_OBJECT(window), "destroy",
                 G_CALLBACK(gtk_main_quit), NULL);

gtk_widget_show_all(window);

gtk_main();

return 0;
}
```

The code example creates simple toolbar example.

Advertisements

```
toolbar = gtk_toolbar_new();
```

The `gtk_toolbar_set_style` function alters the view of toolbar to display either icons only, text only, or both. Passing the `GTK_TOOLBAR_ICONS` constant makes the toolbar show only icons.

```
newTb = gtk_tool_button_new_from_stock(GTK_STOCK_NEW);
```

The `gtk_tool_button_new_from_stock` function creates a new `GtkToolButton` containing the image and text from a stock item.

```
gtk_toolbar_insert(GTK_TOOLBAR(toolbar), new, -1);
```

The `gtk_toolbar_insert` function inserts a `GtkToolItem` into the toolbar at the specified position. If the position is negative, the item is appended to the end of the toolbar.

```
sep = gtk_separator_tool_item_new();  
gtk_toolbar_insert(GTK_TOOLBAR(toolbar), sep, -1);
```

The `gtk_separator_tool_item_new` function creates a new `GtkSeparatorToolItem`. It is inserted into the toolbar with the `gtk_toolbar_insert` function.

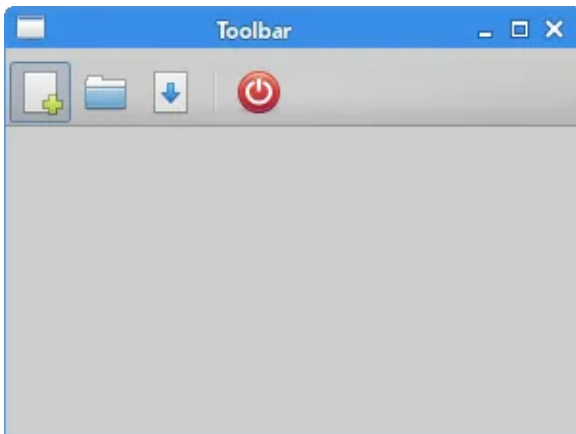


Figure: Toolbar

Undo redo

The following example demonstrates how to inactivate toolbar buttons on the

that all changes are already saved.

undoredo.c

```
#include <gtk/gtk.h>

void undo_redo(GtkWidget *widget, gpointer item) {

    static gint count = 2;
    const gchar *name = gtk_widget_get_name(widget);

    if (g_strcmp0(name, "undo") ) {
        count++;
    } else {
        count--;
    }

    if (count < 0) {
        gtk_widget_set_sensitive(widget, FALSE);
        gtk_widget_set_sensitive(item, TRUE);
    }

    if (count > 5) {
        gtk_widget_set_sensitive(widget, FALSE);
        gtk_widget_set_sensitive(item, TRUE);
    }
}

int main(int argc, char *argv[]) {

    GtkWidget *window;
    GtkWidget *vbox;

    GtkWidget *toolbar;
    GtkToolItem *undo;
    GtkToolItem *redo;
    GtkToolItem *sep;
    GtkToolItem *exit;

    gtk_init(&argc, &argv);

    window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
    gtk_window_set_position(GTK_WINDOW(window), GTK_WIN_POS_CENTER);
    gtk_window_set_default_size(GTK_WINDOW(window), 300, 200);
    gtk_window_set_title(GTK_WINDOW(window), "Undo redo");

    vbox = gtk_vbox_new(FALSE, 0);
    gtk_container_add(GTK_CONTAINER(window), vbox);
```

```
gtk_container_set_border_width(GTK_CONTAINER(toolbar), 2);

undo = gtk_tool_button_new_from_stock(GTK_STOCK_UNDO);
gtk_widget_set_name(GTK_WIDGET(undo), "undo");
gtk_toolbar_insert(GTK_TOOLBAR(toolbar), undo, -1);

redo = gtk_tool_button_new_from_stock(GTK_STOCK_REDO);
gtk_toolbar_insert(GTK_TOOLBAR(toolbar), redo, -1);

sep = gtk_separator_tool_item_new();
gtk_toolbar_insert(GTK_TOOLBAR(toolbar), sep, -1);

exit = gtk_tool_button_new_from_stock(GTK_STOCK_QUIT);
gtk_toolbar_insert(GTK_TOOLBAR(toolbar), exit, -1);

gtk_box_pack_start(GTK_BOX(vbox), toolbar, FALSE, FALSE, 0);

g_signal_connect(G_OBJECT(undo), "clicked",
                 G_CALLBACK(undo_redo), redo);

g_signal_connect(G_OBJECT(redo), "clicked",
                 G_CALLBACK(undo_redo), undo);

g_signal_connect(G_OBJECT(exit), "clicked",
                 G_CALLBACK(gtk_main_quit), NULL);

g_signal_connect(G_OBJECT(window), "destroy",
                 G_CALLBACK(gtk_main_quit), NULL);

gtk_widget_show_all(window);

gtk_main();

return 0;
}
```

Our example creates Undo and Redo buttons from the GTK+ stock resources. After several clicks each of the buttons is inactivated. The buttons are grayed out.

Advertisements

```
if (count < 0) {  
    gtk_widget_set_sensitive(widget, FALSE);  
    gtk_widget_set_sensitive(item, TRUE);  
}  
  
if (count > 5) {  
    gtk_widget_set_sensitive(widget, FALSE);  
    gtk_widget_set_sensitive(item, TRUE);  
}
```

The `gtk_widget_set_sensitive` function is used to activate or inactivate the toolbar buttons.

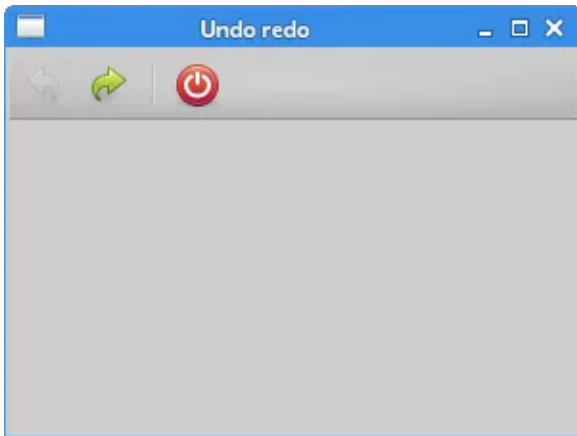


Figure: Undo redo

In this chapter we have covered about menus and toolbars in GTK+.

[Contents](#) [Previous](#) [Next](#)



[report this ad](#)

[Home](#) [Twitter](#) [Github](#) [Subscribe](#) [Privacy](#)

© 2007 - 2023 Jan Bodnar [admin\(at\)zetcode.com](mailto:admin(at)zetcode.com)

